ALTERNATE ASSESSMENT TOOL (AAT) REPORT ON

"Design and Verification of Combinational and Sequence Digital Modules Using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

VISHAL BHIMAPPA SANTI

1BF24EC186

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

CONTENTS

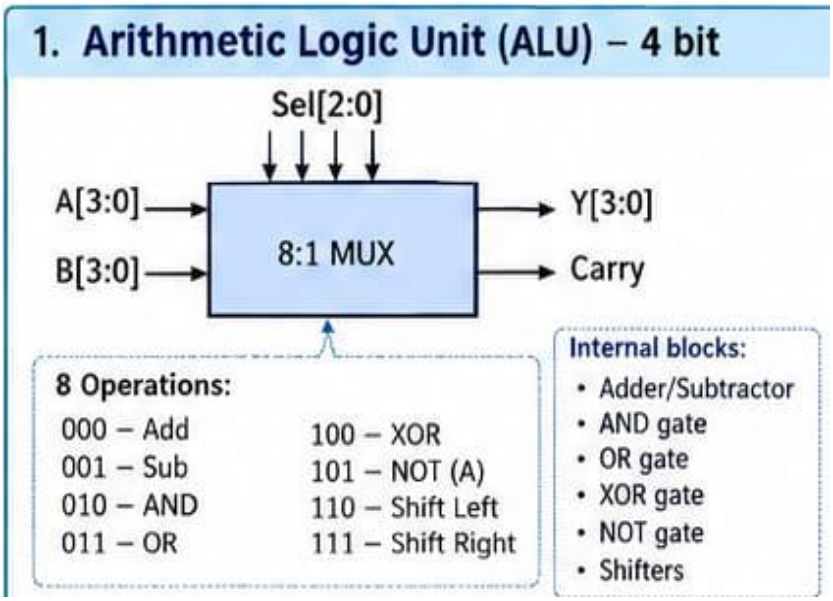
DESIGN, CODE, TESTBENCH

- Arithmetic and logic operations
- 4-bit magnitude comparator
- 4-Bit Parity Generator and Checker
- 4-Bit Barrel Shifter
- Clock-based timer with reset
- 4-bit ring counter
- 4-bit Johnson counter
- Synchronous Counter
- Asynchronous Counter
- 4-bit serial-in, serial-out register
- Parameterized frequency divider
- Mealy FSMs
- Moore FSMs
- LFSR-based sequence generator

• Arithmetic Logic Unit (ALU)

The 4-bit ALU performs eight operations: addition, subtraction, AND, OR, XOR, NOT, left shift, and right shift.

DESIGN:




```
`timescale 1ns/1ps


module alu4(
   input [3:0] a, b,
   input [2:0] sel,
   output reg [3:0] y,
   output reg carry
);


always @(*) begin
   y = 4'b0000;
   carry = 1'b0;


   case(sel)
      3'b000: {carry,y} = {1'b0,a} + {1'b0,b};
      3'b001: y = a - b;
      3'b010: y = a & b;
      3'b011: y = a | b;
      3'b100: y = a ^ b;
      3'b101: y = ~a;
      3'b110: y = a << 1;
      3'b111: y = a >> 1;
      default: y = 4'b0000;
   endcase
end


endmodule
```

TB:

```
`timescale 1ns/1ps


module alu4_tb;


reg [3:0] a, b;
reg [2:0] sel;
wire [3:0] y;
wire carry;
integer i;


alu4 dut(a,b,sel,y,carry);


initial begin
   a = 4'd9;
   b = 4'd5;


   for(i=0; i<8; i=i+1) begin
      sel = i;
      #10;
      $display("sel=%b a=%d b=%d y=%d carry=%b",
            sel,a,b,y,carry);
   end


   $finish;
end


endmodule
```

Working:

For inputs A = 9 and B = 4, greater = 1, while equal = 0 and less = 0.

• 4-bit Comparator

Compares two 4-bit numbers and indicates whether A is greater than, equal to, or less than B

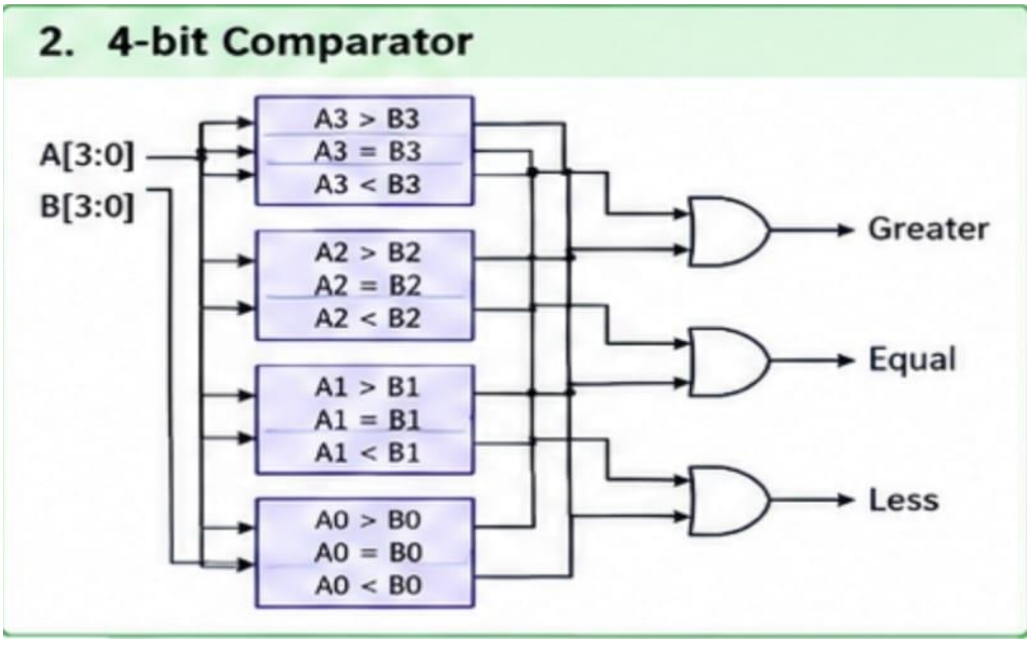
DESIGN:

```
`timescale 1ns/1ps


module comparator4(
   input [3:0] a, b,
   output reg greater,
   output reg equal,
   output reg less
);


always @(*) begin
   greater = (a > b);
   equal   = (a == b);
   less    = (a < b);
end


endmodule
```



TB:

```
`timescale 1ns/1ps


module comparator4_tb;


reg [3:0] a, b;
wire greater, equal, less;


comparator4 dut(a,b,greater,equal,less);


initial begin
   a=9; b=4;  #10;
   $display("A>B: %b A=B: %b A<B: %b",
         greater,equal,less);


   a=6; b=6;  #10;
   $display("A>B: %b A=B: %b A<B: %b",
         greater,equal,less);


   a=2; b=12; #10;
   $display("A>B: %b A=B: %b A<B: %b",
         greater,equal,less);


   $finish;
end


endmodule
```

Working:

The sel input chooses which operation the ALU performs. The always @(*) block updates the output whenever an input changes.

• 4-bit Parity Generator and Checker

Generates even and odd parity bits and checks a received even-parity bit.

DESIGN:

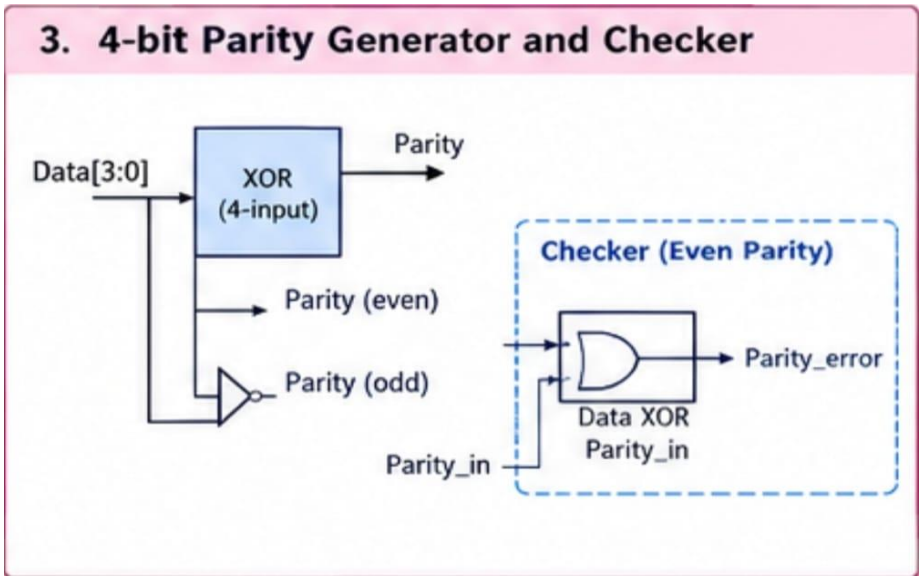
```
`timescale 1ns/1ps


module parity4(
   input [3:0] data_in,
   input parity_in,
   output even_parity_bit,
   output odd_parity_bit,
   output parity_error
);


assign even_parity_bit = ^data_in;
assign odd_parity_bit  = ~(^data_in);


assign parity_error = (^data_in) ^ parity_in;


endmodule
```



TB:

```
`timescale 1ns/1ps


module parity4_tb;


reg [3:0] data_in;
reg parity_in;


wire even_parity_bit;
wire odd_parity_bit;
wire parity_error;


parity4 dut(data_in,parity_in,
        even_parity_bit,
        odd_parity_bit,
        parity_error);


initial begin
   data_in=4'b1011;
   parity_in=1'b1;
   #10;
   $display("Data=%b Even=%b Odd=%b Error=%b",
        data_in,even_parity_bit,
        odd_parity_bit,parity_error);


   parity_in=1'b0;
   #10;
   $display("Wrong parity: Error=%b",parity_error);


   $finish;
end


endmodule
```

Working:

The XOR operator ^ generates the parity bit. The checker produces 1 when the received even-parity bit does not match the expected value.

• 4-bit Barrel Shifter

Shifts the input left or right by 0–3 bit positions.

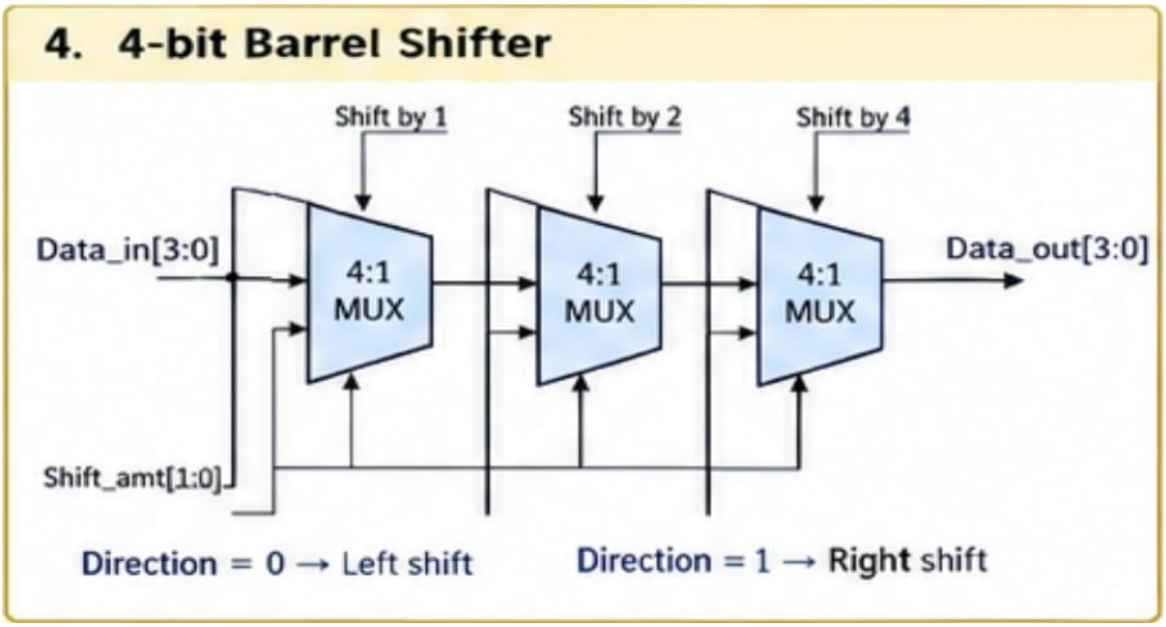
DESIGN:

```
`timescale 1ns/1ps


module barrel_shifter4(
   input [3:0] data_in,
   input [1:0] shift_amt,
   input direction,
   output reg [3:0] data_out
);


always @(*) begin
   if(direction == 1'b0)
      data_out = data_in << shift_amt;
   else
      data_out = data_in >> shift_amt;
end


endmodule
```



TB:

```
`timescale 1ns/1ps


module barrel_shifter4_tb;


reg [3:0] data_in;
reg [1:0] shift_amt;
reg direction;
wire [3:0] data_out;
integer i;


barrel_shifter4 dut(data_in,shift_amt,
                direction,data_out);


initial begin
   data_in=4'b1011;


   direction=0;
   for(i=0; i<4; i=i+1) begin
      shift_amt=i;
      #10;
      $display("Left shift=%d Output=%b",
            shift_amt,data_out);
   end


   direction=1;
   for(i=0; i<4; i=i+1) begin
      shift_amt=i;
      #10;
      $display("Right shift=%d Output=%b",
            shift_amt,data_out);
   end


   $finish;
end


endmodule
```

• Timer Module

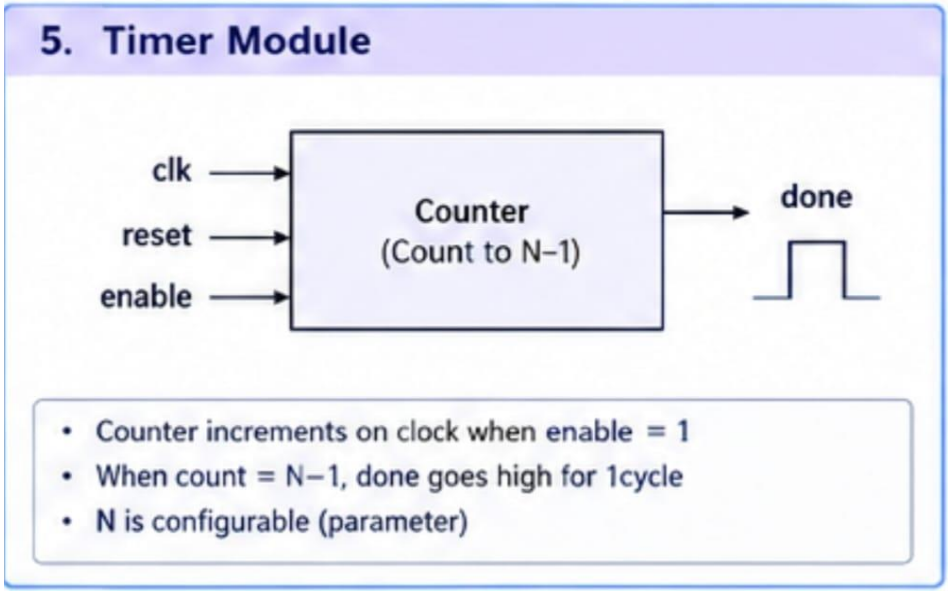
This timer generates a one-clock-cycle done pulse after four enabled clock cycles. The count limit is configurable.

DESIGN:

```
`timescale 1ns/1ps
module timer_module #(
    parameter COUNT_MAX = 4
)(
    input clk,
    input reset,
    input enable,
    output reg done
);

integer count;

always @(posedge clk) begin
    if(reset) begin
        count <= 0;
        done <= 0;
    end
    else if(enable) begin
        if(count == COUNT_MAX-1) begin
            count <= 0;
            done <= 1;
        end
        else begin
            count <= count + 1;
            done <= 0;
        end
    end
    else begin
        done <= 0;
    end
end
endmodule
```



TB:

```
`timescale 1ns/1ps


module timer_module_tb;


reg clk, reset, enable;
wire done;


timer_module #(.COUNT_MAX(4))
   dut(clk,reset,enable,done);


always #5 clk = ~clk;


initial begin
   clk=0;
   reset=1;
   enable=0;


   #12 reset=0;
   enable=1;


   repeat(12) begin
      @(posedge clk);
      #1;
      $display("Time=%0t Done=%b",$time,done);
   end


   $finish;
end


endmodule
```

Working:

The counter increments at each rising clock edge. When it reaches COUNT_MAX-1, it resets and asserts done for one cycle.

• Ring Counter

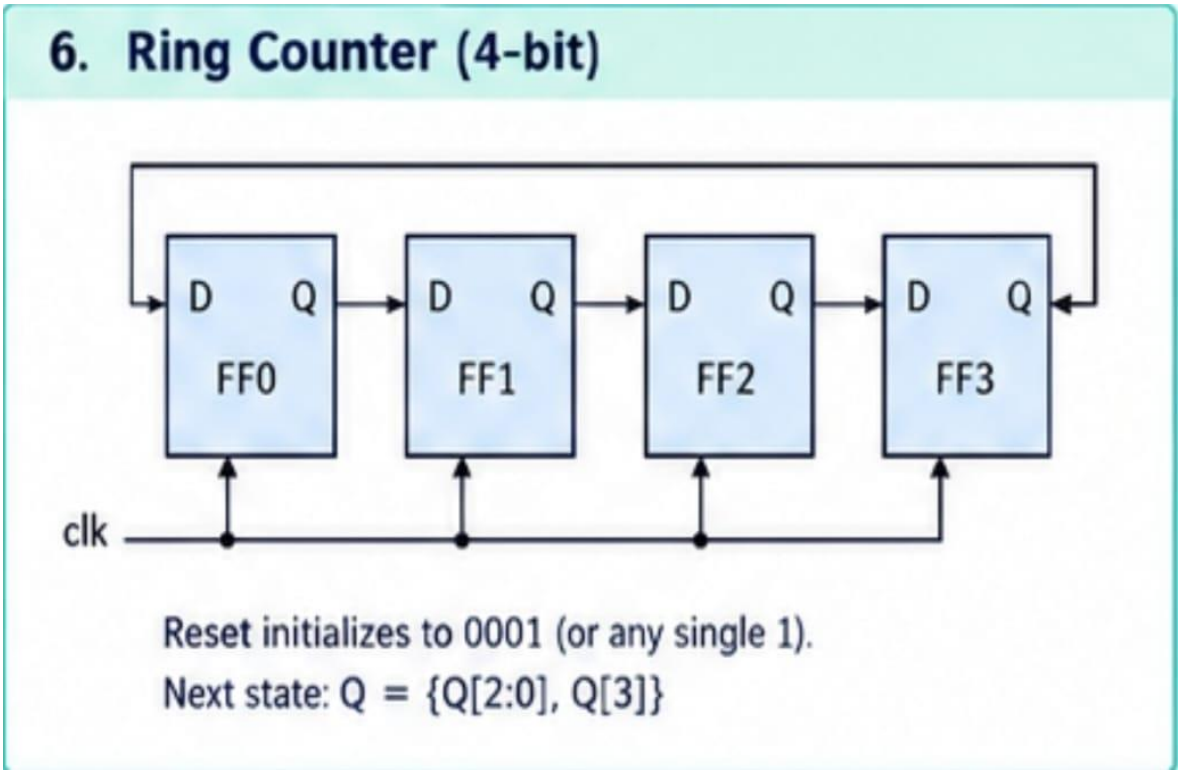
A 4-bit ring counter circulates a single 1 through its four flip-flop positions.

DESIGN:

```
`timescale 1ns/1ps


module ring_counter4(
   input clk,
   input reset,
   input enable,
   output reg [3:0] q
);


always @(posedge clk) begin
   if(reset)
      q <= 4'b0001;
   else if(enable)
      q <= {q[2:0],q[3]};
end
endmodule
```



TB:

```
`timescale 1ns/1ps


module ring_counter4_tb;


reg clk, reset, enable;
wire [3:0] q;


ring_counter4 dut(clk,reset,enable,q);


always #5 clk = ~clk;


initial begin
   clk=0;
   reset=1;
   enable=0;


   #12 reset=0;
   enable=1;


   repeat(8) begin
      @(posedge clk);
      #1;
      $display("Ring Counter=%b",q);
   end


   $finish;
end


endmodule
```

Expected sequence:

0001 → 0010 → 0100 → 1000 → 0001

• Johnson Counter

A 4-bit Johnson counter feeds the inverted most-significant bit back into the input.

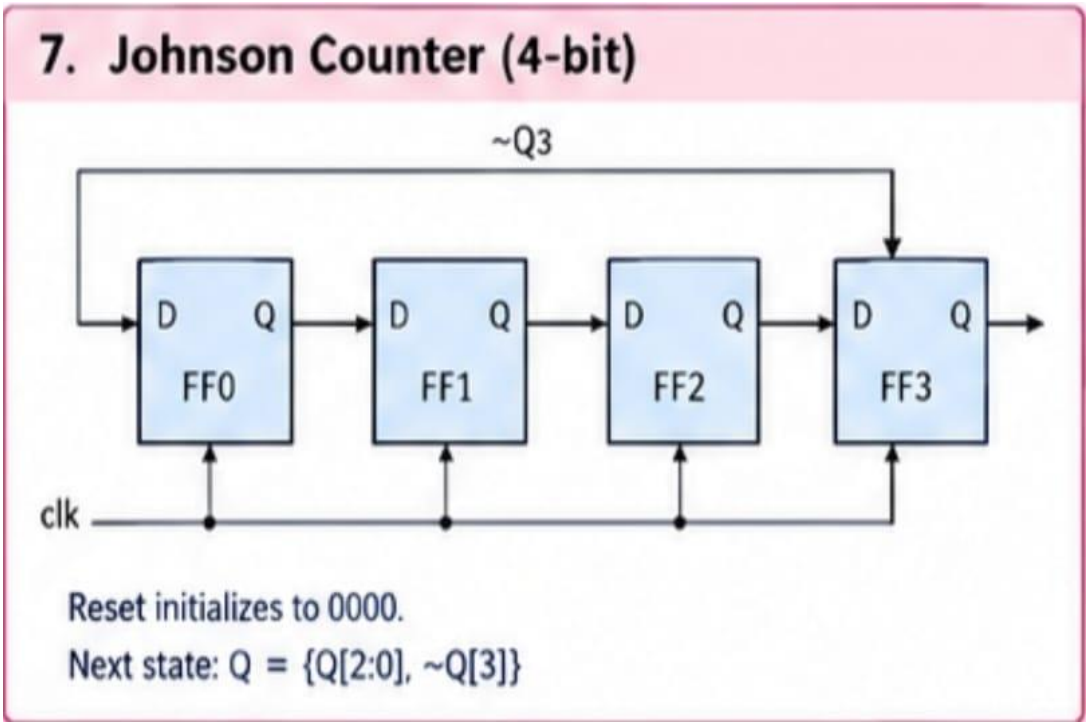
DESIGN:

```
`timescale 1ns/1ps


module johnson_counter4(
   input clk,
   input reset,
   input enable,
   output reg [3:0] q
);


always @(posedge clk) begin
   if(reset)
      q <= 4'b0000;
   else if(enable)
      q <= {q[2:0],~q[3]};
end


endmodule
```



TB:

```
`timescale 1ns/1ps


module johnson_counter4_tb;


reg clk, reset, enable;
wire [3:0] q;


johnson_counter4 dut(clk,reset,enable,q);


always #5 clk = ~clk;


initial begin
   clk=0;
   reset=1;
   enable=0;


   #12 reset=0;
   enable=1;


   repeat(12) begin
      @(posedge clk);
      #1;
      $display("Johnson Counter=%b",q);
   end


   $finish;
end


endmodule
```

Expected sequence:

0000 → 0001 → 0011 → 0111 → 1111 → 1110 → 1100 → 1000 → 0000

- Synchronous/asynchronous counters and shift register

Synchronous Counter

All flip-flops respond to the same clock edge.

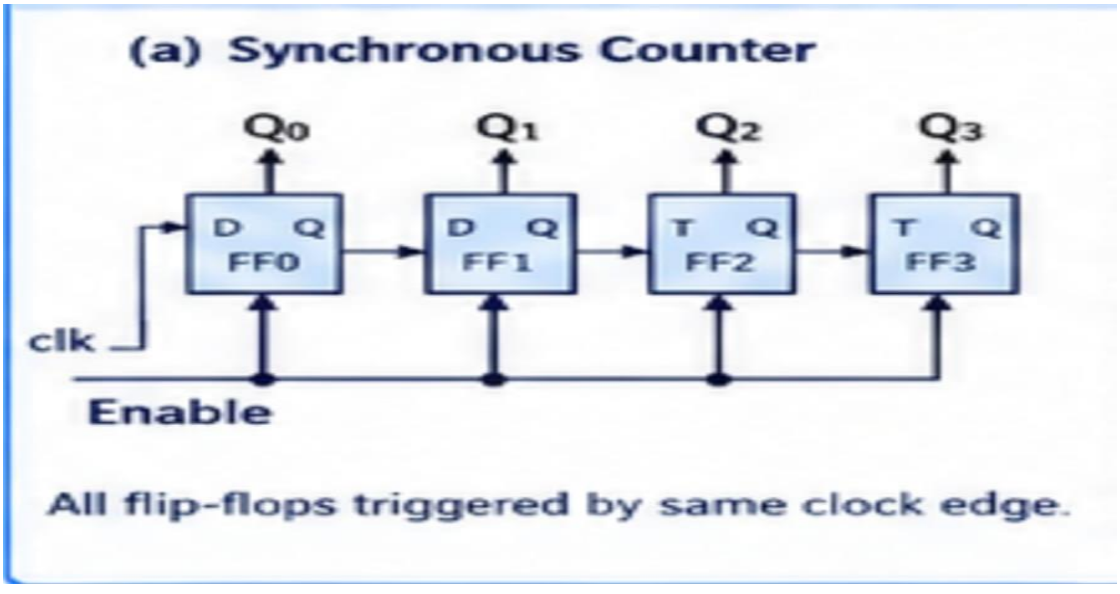
DESIGN:

```
`timescale 1ns/1ps


module synchronous_counter4(
   input clk,
   input reset,
   input enable,
   output reg [3:0] q
);


always @(posedge clk) begin
   if(reset)
      q <= 4'b0000;
   else if(enable)
      q <= q + 1'b1;
end


endmodule
```



TB:

```
`timescale 1ns/1ps


module synchronous_counter4_tb;


reg clk, reset, enable;
wire [3:0] q;


synchronous_counter4 dut(clk,reset,enable,q);


always #5 clk = ~clk;


initial begin
   clk=0;
   reset=1;
   enable=0;


   #12 reset=0;
   enable=1;


   repeat(18) begin
      @(posedge clk);
      #1;
      $display("Synchronous Counter=%b",q);
   end


   $finish;
end


endmodule
```

Working:

The counter increments by one at every rising clock edge when enabled. After 1111, it wraps to 0000.

Asynchronous Counter

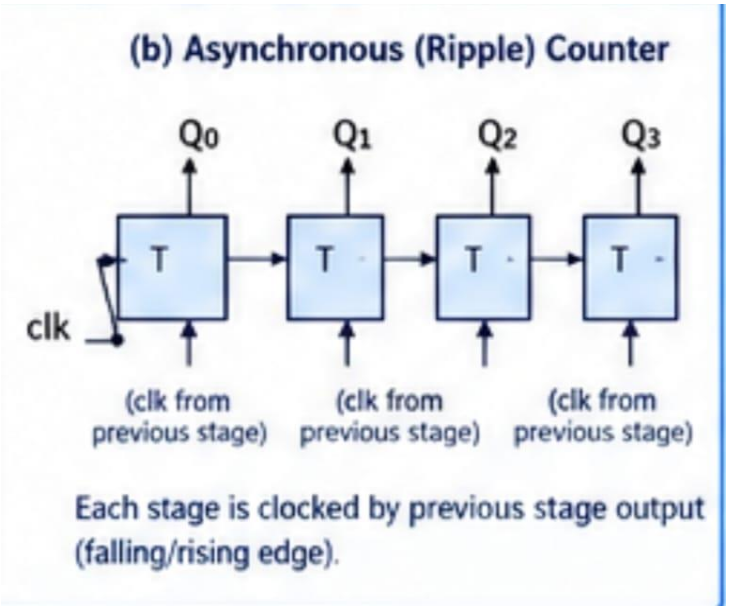
In this ripple counter, each higher bit is clocked by the falling edge of the preceding bit.

DESIGN:

```
`timescale 1ns/1ps
module asynchronous_counter4(
   input clk,
   input reset,
   output reg [3:0] q
);


always @(posedge clk or posedge reset) begin
   if(reset)
      q[0] <= 1'b0;
   else
      q[0] <= ~q[0];
end


always @(negedge q[0] or posedge reset) begin
   if(reset)
      q[1] <= 1'b0;
   else
      q[1] <= ~q[1];
end
always @(negedge q[1] or posedge reset) begin
   if(reset)
      q[2] <= 1'b0;
   else
      q[2] <= ~q[2];
end
always @(negedge q[2] or posedge reset) begin
   if(reset)
      q[3] <= 1'b0;
   else
      q[3] <= ~q[3];
end
endmodule
```



TB:

```
`timescale 1ns/1ps


module asynchronous_counter4_tb;


reg clk, reset;
wire [3:0] q;


asynchronous_counter4 dut(clk,reset,q);


always #5 clk = ~clk;


initial begin
   clk=0;
   reset=1;


   #12 reset=0;


   repeat(20) begin
      @(posedge clk);
      #2;
      $display("Asynchronous Counter=%b",q);
   end


   $finish;
end


endmodule
```

Working:

The first bit toggles with the main clock. The remaining bits toggle through ripple clocking, so transitions can occur at slightly different times.

• 4-bit Shift Register

This is a serial-in, serial-out shift register. New bits enter at q[0], and the serial output is taken from q[3].

DESIGN:

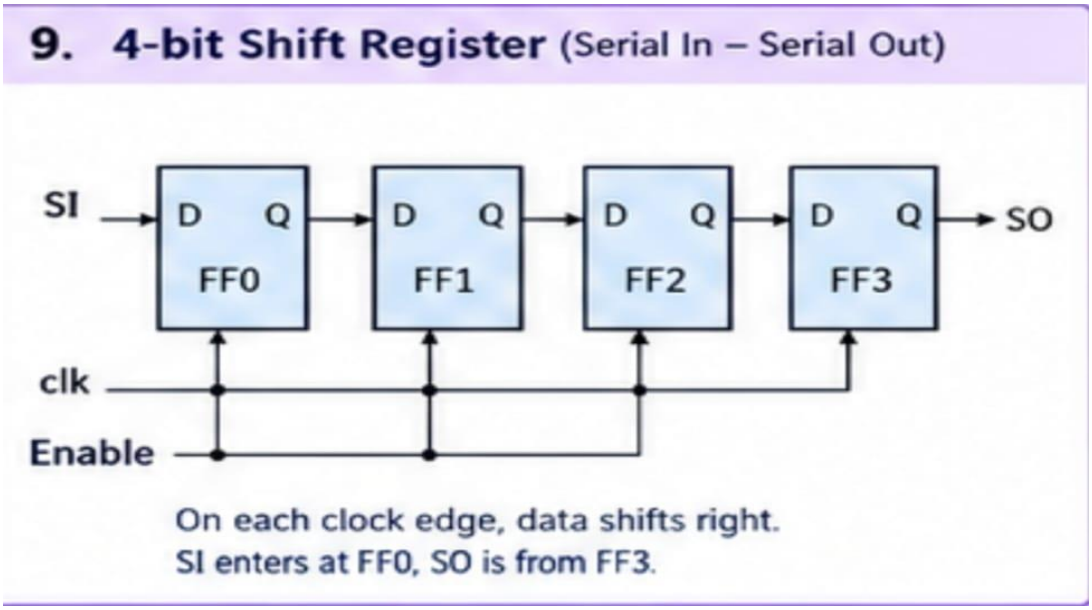
```
`timescale 1ns/1ps


module shift_register4(
   input clk,
   input reset,
   input enable,
   input serial_in,
   output reg [3:0] q,
   output serial_out
);


always @(posedge clk) begin
   if(reset)
      q <= 4'b0000;
   else if(enable)
      q <= {q[2:0],serial_in};
end


assign serial_out = q[3];


endmodule
```



TB:

```
`timescale 1ns/1ps
module shift_register4_tb;
reg clk, reset, enable, serial_in;
wire [3:0] q;
wire serial_out;

shift_register4 dut(
   clk,reset,enable,serial_in,q,serial_out
);

always #5 clk = ~clk;

initial begin
   clk=0;
   reset=1;
   enable=0;
   serial_in=0;

   #12 reset=0;
   enable=1;

   serial_in=1; @(posedge clk); #1;
   $display("Q=%b Serial Out=%b",q,serial_out);

   serial_in=0; @(posedge clk); #1;
   $display("Q=%b Serial Out=%b",q,serial_out);

   serial_in=1; @(posedge clk); #1;
   $display("Q=%b Serial Out=%b",q,serial_out);

   serial_in=1; @(posedge clk); #1;
   $display("Q=%b Serial Out=%b",q,serial_out);

   $finish;
end
endmodule
```

• Divide-by-N Counter

This counter divides the input clock frequency by a selected even number N. For example, N=8 produces an output clock with one-eighth of the input frequency.

DESIGN:

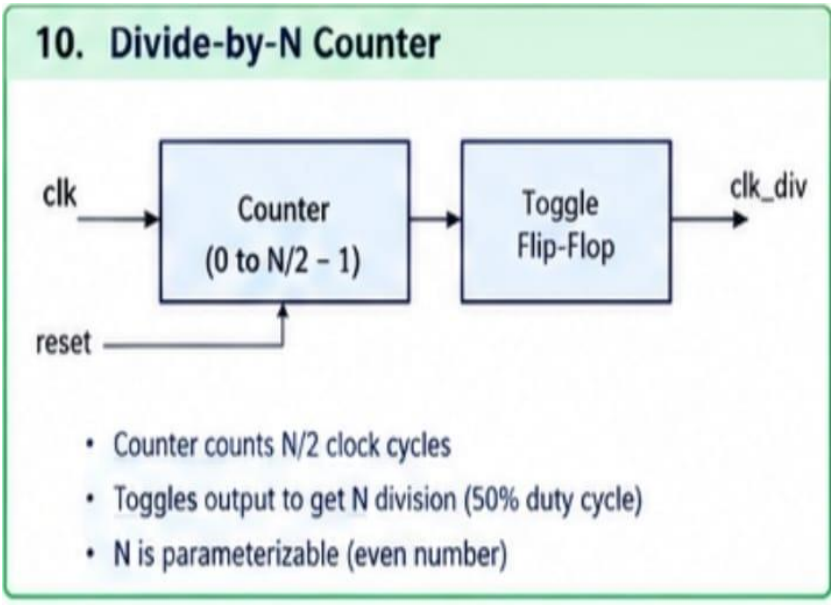
```
`timescale 1ns/1ps


module divide_by_n #(
   parameter N = 8
)(
   input clk,
   input reset,
   output reg clk_div
);


integer count;
localparam HALF = (N < 2) ? 1 : (N/2);


always @(posedge clk) begin
   if(reset) begin
      count <= 0;
      clk_div <= 1'b0;
   end
   else if(count >= HALF-1) begin
      count <= 0;
      clk_div <= ~clk_div;
   end
   else begin
      count <= count + 1;
   end
end


endmodule
```



TB:

```
`timescale 1ns/1ps

module divide_by_n_tb;


reg clk, reset;

wire clk_div;


divide_by_n #(.N(8)) dut(clk,reset,clk_div);


always #5 clk = ~clk;


initial begin

   clk=0;

   reset=1;


   #12 reset=0;


   repeat(30) begin

      @(posedge clk);

      #1;

      $display("Time=%0t Divided Clock=%b",

            $time,clk_div);

   end


   $finish;

end

endmodule
```

Working:

The internal counter counts four input clock cycles for N=8, then toggles the output. Two toggles complete one output-clock period, giving f

out=fin/8.

This implementation is intended for even values of N. Odd values need a different approach if an exact divide ratio and a 50% duty cycle are required.

• Mealy FSM — Sequence Detector for 101

A Mealy FSM generates its output based on the current state and current input. This design detects the serial pattern 101, including overlapping occurrences.

DESIGN:

```
`timescale 1ns/1ps


module mealy_101_detector(
   input clk,
   input reset,
   input din,
   output reg detected
);


reg [1:0] state;


localparam S0  = 2'b00,
         S1  = 2'b01,
         S10 = 2'b10;


always @(posedge clk) begin
   if(reset) begin
      state <= S0;
      detected <= 1'b0;
   end
   else begin
      detected <= 1'b0;

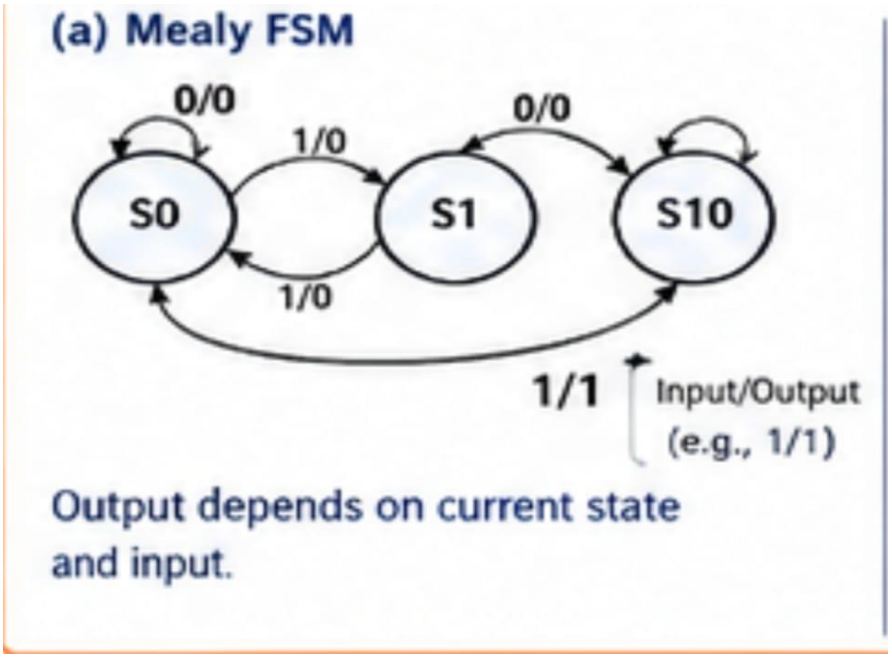

      case(state)
         S0: begin
            if(din) state <= S1;
            else    state <= S0;
         end


         S1: begin
            if(din) state <= S1;
            else    state <= S10;
```

```
        end


        S10: begin
          if(din) begin
            state <= S1;
            detected <= 1'b1;
          end
          else begin
            state <= S0;
          end
        end

        default: begin
          state <= S0;
          detected <= 1'b0;
        end
      endcase
    end
end
endmodule
```



TB:

```
`timescale 1ns/1ps


module mealy_101_detector_tb;


reg clk, reset, din;
wire detected;
integer i;
reg [7:0] bits;


mealy_101_detector dut(clk,reset,din,detected);


always #5 clk = ~clk;


initial begin
   clk=0;
   reset=1;
   din=0;
   bits=8'b00101101;


   #12 reset=0;


   for(i=7; i>=0; i=i-1) begin
      din=bits[i];
      @(posedge clk);
      #1;
      $display("Input=%b Detected=%b",din,detected);
   end


   $finish;
end
endmodule
```

Working:

The FSM remembers whether it has received the first bit (1) or the first two bits (10). When the next input completes 101, detected becomes high for one clock cycle.

• Moore FSM — Sequence Detector for 101

A Moore FSM produces its output based only on the current state. The output is asserted when the machine enters the state representing the detected pattern.

DESIGN:

```
`timescale 1ns/1ps
module moore_101_detector(
   input clk,
   input reset,
   input din,
   output reg detected
);

reg [1:0] state;

localparam S0   = 2'b00,
         S1   = 2'b01,
         S10  = 2'b10,
         S101 = 2'b11;

always @(posedge clk) begin
   if(reset) begin
      state <= S0;
      detected <= 1'b0;
   end
   else begin
      case(state)
         S0:
            if(din) state <= S1;
            else    state <= S0;

         S1:
            if(din) state <= S1;
            else    state <= S10;

         S10:
            if(din) state <= S101;
```

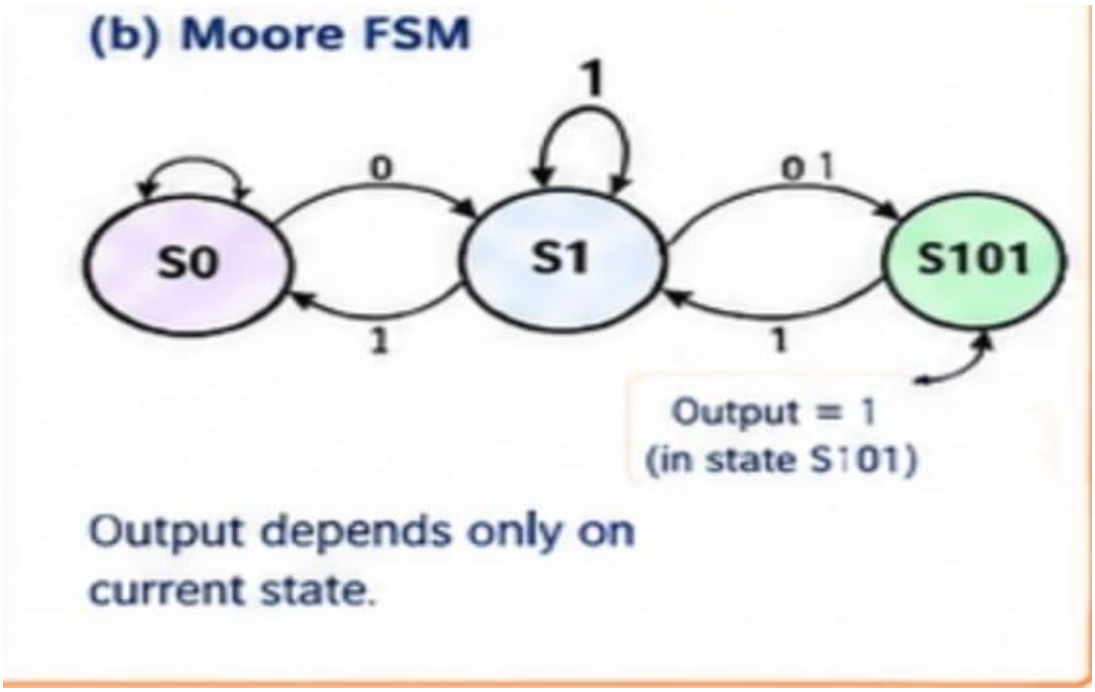
```
            else    state <= S0;


          S101:
            if(din) state <= S1;
            else    state <= S10;


          default: state <= S0;
       endcase


       detected <= (state == S101);
    end
end


endmodule
```



TB:

```
`timescale 1ns/1ps


module moore_101_detector_tb;


reg clk, reset, din;
wire detected;
integer i;
reg [7:0] bits;


moore_101_detector dut(clk,reset,din,detected);


always #5 clk = ~clk;


initial begin
   clk=0;
   reset=1;
   din=0;
   bits=8'b00101101;


   #12 reset=0;


   for(i=7; i>=0; i=i-1) begin
      din=bits[i];
      @(posedge clk);
      #1;
      $display("Input=%b Detected=%b",din,detected);
   end


   $finish;
end
endmodule
```

Working:

The machine moves through S0, S1, S10, and S101. The output is high when the current state is S101, so detection appears one clock later than in the corresponding Mealy implementation

- Random Sequence Generator

4-bit LFSR (Linear Feedback Shift Register)

An LFSR generates a repeating pseudo-random binary sequence using shift-register feedback and XOR logic. It is useful for sequence generation and testing digital circuits.

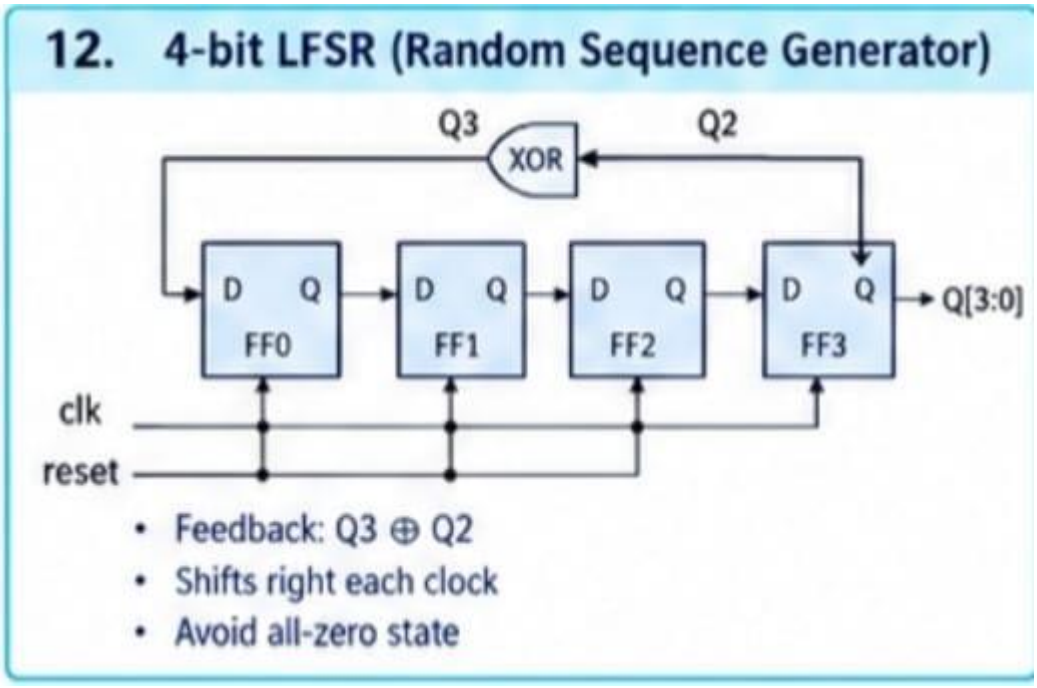
DESIGN:

```
`timescale 1ns/1ps


module lfsr4(
   input clk,
   input reset,
   input enable,
   output reg [3:0] q
);


always @(posedge clk) begin
   if(reset)
      q <= 4'b0001;
   else if(enable)
      q <= {q[2:0], q[3] ^ q[2]};
end


endmodule
```



TB:

```
`timescale 1ns/1ps


module lfsr4_tb;


reg clk, reset, enable;
wire [3:0] q;


lfsr4 dut(clk,reset,enable,q);


always #5 clk = ~clk;


initial begin
   clk=0;
   reset=1;
   enable=0;


   #12 reset=0;
   enable=1;


   repeat(20) begin
      @(posedge clk);
      #1;
      $display("LFSR Output=%b",q);
   end


   $finish;
end
endmodule
```

Working:

Reset initializes the register to 0001.

At each rising clock edge, the register shifts one position.

The XOR of q[3] and q[2] supplies the feedback bit.

The register generates a repeating pseudo-random sequence.The all-zero state should be avoided in an XOR-feedback LFSR because it remains locked at zero.